

1. Dada la siguiente signatura

Signatura corregida

```
signature ESTUDIANTE = sig
  type dataEstudiante
  exception StudentError

  val empty : dataEstudiante

  val putNombre : string * dataEstudiante -> dataEstudiante
  val putApellido : string * dataEstudiante ->
dataEstudiante
  val putID : string * dataEstudiante -> dataEstudiante
  val putGenero : string * dataEstudiante -> dataEstudiante
  val putAgnoAcademico : int * dataEstudiante ->
dataEstudiante
  val putIndice : real * dataEstudiante -> dataEstudiante

  val getNombre : dataEstudiante -> string
  val getApellido : dataEstudiante -> string
  val getID : dataEstudiante -> string
  val getGenero : dataEstudiante -> char
  val getAgnoAcademico : dataEstudiante -> int
  val getIndice : dataEstudiante -> real
end;
```

- a. Implemente la signatura dada más arriba usando una estructura que encapsule una lista de asociación (una lista de pares ordenados). Para cada entrada en la lista, el primer componente designa el nombre de la propiedad y el segundo, el valor de esta. Provea casos de prueba para cada una de las funciones

```
signature ESTUDIANTE = sig
  type dataEstudiante
  exception StudentError
  val empty : dataEstudiante
  val putNombre : string * dataEstudiante ->
dataEstudiante
  val putApellido : string * dataEstudiante ->
dataEstudiante
  val putID : string * dataEstudiante -> dataEstudiante
  val putGenero : string * dataEstudiante ->
dataEstudiante
  val putAgnoAcademico : int * dataEstudiante ->
dataEstudiante
  val putIndice : real * dataEstudiante -> dataEstudiante
  val getNombre : dataEstudiante -> string
```

```
    val getApellido : dataEstudiante -> string
    val getID : dataEstudiante -> string
    val getGenero : dataEstudiante -> char
    val getAgnoAcademico : dataEstudiante -> int
    val getIndice : dataEstudiante -> real
end;

structure EstudianteLista :> ESTUDIANTE = struct
    datatype valor =
        VString of string
        | VChar of char
        | VInt of int
        | VReal of real
    type dataEstudiante = (string * valor) list
    exception StudentError
    val empty = []
    fun exists _ [] = false
      | exists key ((k, _) :: xs) = (key = k) orelse exists
key xs
    fun insert (k, v, datos) =
        if exists k datos then raise StudentError
        else (k, v) :: datos
    fun buscar _ [] = raise StudentError
      | buscar key ((k, v) :: xs) =
        if key = k then v else buscar key xs
    fun oneChar s =
        if String.size s = 1 then String.sub (s, 0)
        else raise StudentError
    fun putNombre (s, d) = insert ("nombre", VString s, d)
    fun putApellido (s, d) = insert ("apellido", VString s,
d)
    fun putID (s, d) = insert ("id", VString s, d)
    fun putGenero (s, d) = insert ("genero", VChar (oneChar
s), d)
    fun putAgnoAcademico (n, d) = insert ("agnoAcademico",
VInt n, d)
    fun putIndice (r, d) = insert ("indice", VReal r, d)
    fun getNombre d =
        case buscar "nombre" d of
            VString s => s
          | _ => raise StudentError
    fun getApellido d =
        case buscar "apellido" d of
            VString s => s
          | _ => raise StudentError
    fun getID d =
        case buscar "id" d of
            VString s => s
          | _ => raise StudentError
```



```
fun getGenero d =
  case buscar "genero" d of
    VChar c => c
  | _ => raise StudentError
fun getAñoAcademico d =
  case buscar "añoAcademico" d of
    VInt n => n
  | _ => raise StudentError
fun getIndice d =
  case buscar "indice" d of
    VReal r => r
  | _ => raise StudentError
end;

val el0 = EstudianteLista.empty;
val el1 = EstudianteLista.putNombre ("Xalven", el0);
val el2 = EstudianteLista.putApellido ("Drayk", el1);
val el3 = EstudianteLista.putID ("QZ7-4812", el2);
val el4 = EstudianteLista.putGenero ("M", el3);
val el5 = EstudianteLista.putAñoAcademico (5, el4);
val el6 = EstudianteLista.putIndice (3.72, el5);

val pruebaLNombre1 = EstudianteLista.getNombre el6;
val pruebaLApellido1 = EstudianteLista.getApellido el6;
val pruebaLID1 = EstudianteLista.getID el6;
val pruebaLGenero1 = EstudianteLista.getGenero el6;
val pruebaLAño1 = EstudianteLista.getAñoAcademico el6;
val pruebaLIndice1 = EstudianteLista.getIndice el6;

val elA0 = EstudianteLista.empty;
val elA1 = EstudianteLista.putNombre ("Nyvora", elA0);
val elA2 = EstudianteLista.putApellido ("Selq", elA1);
val elA3 = EstudianteLista.putID ("LM4-9305", elA2);
val elA4 = EstudianteLista.putGenero ("F", elA3);
val elA5 = EstudianteLista.putAñoAcademico (2, elA4);
val elA6 = EstudianteLista.putIndice (2.89, elA5);

val pruebaLNombre2 = EstudianteLista.getNombre elA6;
val pruebaLApellido2 = EstudianteLista.getApellido elA6;
val pruebaLID2 = EstudianteLista.getID elA6;
val pruebaLGenero2 = EstudianteLista.getGenero elA6;
val pruebaLAño2 = EstudianteLista.getAñoAcademico elA6;
val pruebaLIndice2 = EstudianteLista.getIndice elA6;

val errorListaRepetidoNombre =
  (EstudianteLista.putNombre ("OtroNombre", el6);
  "fallo")
  handle EstudianteLista.StudentError => "ok";

val errorListaRepetidoID =
```



```
(EstudianteLista.putID ("AA0-0000", eIA6); "fallo")  
handle EstudianteLista.StudentError => "ok";
```



```
val errorListaGeneroInvalido =  
    (EstudianteLista.putGenero ("Masculino",  
EstudianteLista.empty); "fallo")  
    handle EstudianteLista.StudentError => "ok";
```

En el inciso 1a hice una estructura llamada EstudianteLista que guarda la información del estudiante usando una lista de asociación. Eso significa que cada dato se guarda como un par, donde una parte dice qué propiedad es, por ejemplo "nombre" o "id", y la otra parte guarda el valor de esa propiedad. Como no todos los valores son del mismo tipo, hice un tipo auxiliar llamado valor, para poder guardar string, char, int y real dentro de la misma lista.



Las funciones putNombre, putApellido, putID, putGenero, putAgnoAcademico y putIndice sirven para agregar cada dato del estudiante. Antes de insertar el dato, el programa revisa si esa propiedad ya existe en la lista. Si ya existe, entonces lanza la excepción StudentError, porque el ejercicio dice que cada función put solo puede usarse una vez por propiedad.

Las funciones getNombre, getApellido, getID, getGenero, getAgnoAcademico y getIndice sirven para buscar y devolver cada dato. Estas funciones revisan la lista, encuentran la propiedad correspondiente y devuelven el valor en el tipo correcto. En el caso de putGenero, como el enunciado lo recibe como string pero getGenero debe devolver char, lo que hice fue tomar una cadena de un solo carácter y convertirla a char. Si el texto tiene más de un carácter, también se lanza StudentError.

Las pruebas muestran que el código funciona bien cuando se insertan los datos correctamente, y también comprueban que se produce error cuando se intenta repetir una propiedad o poner un género con más de un carácter.

b. Implemente la signature dada más arriba usando usando una estructura que encapsule un registro (record) de SML. Provea casos de prueba para cada una de las funciones



Nota: Las funciones put solo pueden ser llamadas una vez. Si son llamadas ma's de una vez, el codigo debe lanzar una excepcion.

```
signature SECUENCIA = sig  
    type 'a sec
```

```
    val empty : 'a sec
    val addFront : 'a * 'a sec -> 'a sec
    val addBack : 'a * 'a sec -> 'a sec
    val seqAppend : 'a sec * 'a sec -> 'a sec
    val secToList : 'a sec -> 'a list
end;

structure Secuencia :> SECUENCIA = struct
    datatype 'a secuencia =
        Vacia
        | Uno of 'a
        | Juntar of 'a secuencia * 'a secuencia
    type 'a sec = 'a secuencia
    val empty = Vacia
    fun addFront (x, Vacia) = Uno x
      | addFront (x, s) = Juntar (Uno x, s)
    fun addBack (x, Vacia) = Uno x
      | addBack (x, s) = Juntar (s, Uno x)
    fun seqAppend (Vacia, s) = s
      | seqAppend (s, Vacia) = s
      | seqAppend (s1, s2) = Juntar (s1, s2)
    fun secToList Vacia = []
      | secToList (Uno x) = [x]
      | secToList (Juntar (s1, s2)) = secToList s1 @ secToList
      s2
end;

val s0 = Secuencia.empty;
val s1 = Secuencia.addFront ("Krynn", s0);
val s2 = Secuencia.addFront ("Velor", s1);
val s3 = Secuencia.addBack ("Morth", s2);
val s4 = Secuencia.addBack ("Zyra", Secuencia.empty);
val s5 = Secuencia.seqAppend (s3, s4);

val pruebaSecVacía = Secuencia.secToList s0;
val pruebaSecFront = Secuencia.secToList s2;
val pruebaSecBack = Secuencia.secToList s3;
val pruebaSecAppend = Secuencia.secToList s5;

val n0 = Secuencia.empty;
val n1 = Secuencia.addFront (47, n0);
val n2 = Secuencia.addBack (103, n1);
val n3 = Secuencia.addFront (8, n2);
val n4 = Secuencia.addBack (256, n3);

val pruebaNum1 = Secuencia.secToList n1;
val pruebaNum2 = Secuencia.secToList n2;
val pruebaNum3 = Secuencia.secToList n3;
val pruebaNum4 = Secuencia.secToList n4;
```

5.1



En el inciso 1b hice otra estructura llamada EstudianteRegistro, pero esta vez usando un record de SML. Aquí la información del estudiante está organizada en campos fijos como nombre, apellido, id, genero, agnoAcademico e indice.

Usé option en cada campo porque así puedo representar dos situaciones: cuando un dato todavía no ha sido colocado y cuando ya tiene valor. Si el campo no tiene nada, está en NONE. Si ya tiene un dato, está en SOME valor.



Las funciones put revisan si el campo está vacío. Si está en NONE, entonces colocan el nuevo valor y devuelven un nuevo registro actualizado. Si ya tenía un valor, entonces lanzan la excepción StudentError, porque el enunciado no permite llenar el mismo campo dos veces.

Las funciones get hacen lo contrario: revisan el registro y sacan el valor del campo. Si el campo todavía está vacío, también lanzan la excepción. Igual que en el ejercicio anterior, en putGenero convierto el string a char, siempre que tenga un solo carácter.



Las pruebas de este inciso sirven para demostrar que la estructura funciona correctamente con datos normales y también que detecta errores cuando se repite un campo o cuando el género no cumple con el formato esperado.

2. Defina una estructura y la correspondiente signature de el tipo de dato abstracto que se da a continuación

Estructura: Una secuencia defina por:

Una secuencia is o bien vacía, o bien es una secuencia con un solo elemento, o bien es una secuencia seguida por otra secuencia.

En el inciso 1b hice otra estructura llamada EstudianteRegistro, pero esta vez usando un record de SML. Aquí la información del estudiante está organizada en campos fijos como nombre, apellido, id, genero, agnoAcademico e indice.



Usé option en cada campo porque así puedo representar dos situaciones: cuando un dato todavía no ha sido colocado y cuando ya tiene valor. Si el campo no tiene nada, está en NONE. Si ya tiene un dato, está en SOME valor.

Las funciones put revisan si el campo está vacío. Si está en NONE, entonces colocan el nuevo valor y devuelven un nuevo

registro actualizado. Si ya tenía un valor, entonces lanzan la excepción `StudentError`, porque el enunciado no permite llenar el mismo campo dos veces.



Las funciones `get` hacen lo contrario: revisan el registro y sacan el valor del campo. Si el campo todavía está vacío, también lanzan la excepción. Igual que en el ejercicio anterior, en `putGenero` convierto el string a char, siempre que tenga un solo carácter.

Las pruebas de este inciso sirven para demostrar que la estructura funciona correctamente con datos normales y también que detecta errores cuando se repite un campo o cuando el género no cumple con el formato esperado.



```
signature SECUENCIA = sig
  type 'a sec
  val empty : 'a sec
  val addFront : 'a * 'a sec -> 'a sec
  val addBack : 'a * 'a sec -> 'a sec
  val seqAppend : 'a sec * 'a sec -> 'a sec
  val secToList : 'a sec -> 'a list
end;

structure Secuencia :> SECUENCIA = struct
  datatype 'a secuencia =
    Vacia
    | Uno of 'a
    | Juntar of 'a secuencia * 'a secuencia

  type 'a sec = 'a secuencia

  val empty = Vacia

  fun addFront (x, Vacia) = Uno x
    | addFront (x, s) = Juntar (Uno x, s)

  fun addBack (x, Vacia) = Uno x
    | addBack (x, s) = Juntar (s, Uno x)

  fun seqAppend (Vacia, s) = s
    | seqAppend (s, Vacia) = s
    | seqAppend (s1, s2) = Juntar (s1, s2)

  fun secToList Vacia = []
    | secToList (Uno x) = [x]
    | secToList (Juntar (s1, s2)) = secToList s1 @ secToList
s2
end;
```



```
val s0 = Secuencia.empty;
val s1 = Secuencia.addFront ("Krynn", s0);
val s2 = Secuencia.addFront ("Velor", s1);
val s3 = Secuencia.addBack ("Morth", s2);
val s4 = Secuencia.addBack ("Zyra", Secuencia.empty);
val s5 = Secuencia.seqAppend (s3, s4);

val pruebaSecVacía = Secuencia.secToList s0;
val pruebaSecFront = Secuencia.secToList s2;
val pruebaSecBack = Secuencia.secToList s3;
val pruebaSecAppend = Secuencia.secToList s5;

val n0 = Secuencia.empty;
val n1 = Secuencia.addFront (47, n0);
val n2 = Secuencia.addBack (103, n1);
val n3 = Secuencia.addFront (8, n2);
val n4 = Secuencia.addBack (256, n3);

val pruebaNum1 = Secuencia.secToList n1;
val pruebaNum2 = Secuencia.secToList n2;
val pruebaNum3 = Secuencia.secToList n3;
val pruebaNum4 = Secuencia.secToList n4;
```

- a. Defina un tipo de dato (algebraico) llamado '*a* *secuencia*' que represente la definición de secuencia dada más arriba
- Funciones:

Para representar la secuencia, definí un tipo de dato algebraico llamado *secuencia*. Lo hice de esa forma porque el enunciado dice que una secuencia puede estar vacía, puede tener un solo elemento o puede estar compuesta por una secuencia seguida de otra. Por eso usé tres constructores: uno para la secuencia vacía, otro para una secuencia con un único valor y otro para unir dos secuencias. De esa manera, el tipo refleja exactamente la definición dada en el ejercicio.

- b. Defina una función *addFront* : '*a* * '*a* *sec* -> '*a* *sec* que agregue un elemento al inicio de la secuencia *en tiempo constante*

La función *addFront* fue definida para agregar un elemento al inicio de la secuencia. Si la secuencia está vacía, simplemente crea una secuencia con ese único elemento. Si la secuencia ya contiene datos, entonces construye una nueva secuencia colocando primero el nuevo elemento y después la secuencia original. Esta operación se hace de forma directa, sin recorrer toda la estructura.

- c. Defina una función *addBack* : $'a * 'a \text{ sec} \rightarrow 'a \text{ sec}$ que agregue un elemento al final de la secuencia *en tiempo constante*

La función *addBack* fue creada para insertar un elemento al final de la secuencia. Si la secuencia está vacía, el resultado es una secuencia con un solo elemento. Si la secuencia ya existe, entonces se forma una nueva secuencia donde primero va la secuencia original y luego el nuevo elemento al final. Así se cumple lo pedido en el ejercicio de agregar por detrás.



- d. Defina una función *seqAppend* : $'a \text{ sec} * 'a \text{ sec} \rightarrow 'a \text{ sec}$ que tome dos secuencias y agregue la primera la segunda *en tiempo constante*

La función *seqAppend* se encarga de unir dos secuencias en una sola. Si una de ellas está vacía, la función devuelve la otra, porque no tiene sentido juntar con una secuencia sin elementos. Si las dos tienen contenido, entonces se construye una nueva secuencia que representa a la primera seguida de la segunda. Esta función también se hace de forma sencilla usando la misma estructura recursiva del tipo definido.



- e. Defina una función *secToList* : $'a \text{ sec} \rightarrow 'a \text{ list}$ que tome una secuencia y produzca la lista correspondiente

La función *secToList* convierte una secuencia en una lista normal de SML. Para lograrlo, analiza cada caso del tipo de dato. Si la secuencia está vacía, devuelve una lista vacía. Si tiene un solo elemento, devuelve una lista que contiene ese elemento. Si la secuencia está formada por dos partes unidas, convierte ambas partes a lista y luego las concatena. Así se obtiene una lista con el mismo orden de los elementos de la secuencia.



- f. Proponga casos de prueba que demuestre el funcionamiento de cada una de las funciones.

Los casos de prueba fueron propuestos para demostrar que cada función funciona correctamente. Probé la secuencia vacía, la inserción al frente, la inserción al final, la unión de secuencias y la conversión a lista. También usé ejemplos con cadenas y con números enteros para demostrar que la estructura puede trabajar con diferentes tipos de datos. Con estas pruebas se puede observar claramente que las operaciones hacen lo que el ejercicio solicita.



Index of comments

- 3.1 No se muestra la efectividad de los casos de prueba (-2.00 pts)
- 5.1 No se muestra la ejecución de los casos de prueba (-2.00 pts)
- 8.1 No se muestran los resultados de los casos de prueba (-2.00 pts)